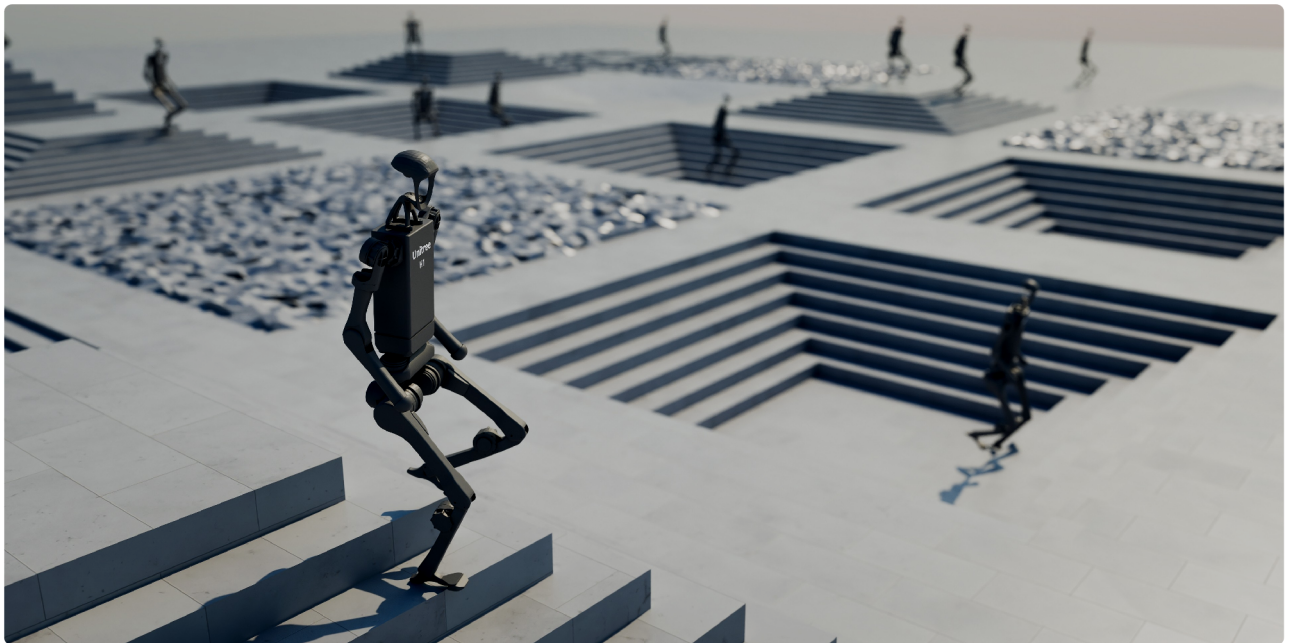


Welcome to Isaac Lab!

Contents

- Welcome to Isaac Lab!
- License
- Citation
- Acknowledgement
- Table of Contents
- Indices and tables



Isaac Lab is a unified and modular framework for robot learning that aims to simplify common workflows in robotics research (such as reinforcement learning, learning from demonstrations, and motion planning). It is built on **NVIDIA Isaac Sim** to leverage the latest simulation capabilities for photo-realistic scenes, and fast and efficient simulation.

The core objectives of the framework are:

- **Modularity:** Easily customize and add new environments, robots, and sensors.
- **Agility:** Adapt to the changing needs of the community.

- **Openness:** Remain open-sourced to allow the community to contribute and extend the framework.
- **Batteries-included:** Include a number of environments, sensors, and tasks that are ready to use.

Key features available in Isaac Lab include fast and accurate physics simulation provided by PhysX, tiled rendering APIs for vectorized rendering, domain randomization for improving robustness and adaptability, and support for running in the cloud.

Additionally, Isaac Lab provides a variety of environments, and we are actively working on adding more environments to the list. These include classic control tasks, fixed-arm and dexterous manipulation tasks, legged locomotion tasks, and navigation tasks. A complete list is available in the [environments](#) section.

Isaac lab is developed with specific robot assets that are now **Batteries-included** as part of the platform and are ready to learn! These robots include...

- **Classic** Cartpole, Humanoid, Ant
- **Fixed-Arm and Hands:** UR10, Franka, Allegro, Shadow Hand
- **Quadrupeds:** Anybotics Anymal-B, Anymal-C, Anymal-D, Unitree A1, Unitree Go1, Unitree Go2, Boston Dynamics Spot
- **Humanoids:** Unitree H1, Unitree G1
- **Quadcopter:** Crazyflie

The platform is also designed so that you can add your own robots! Please refer to the [How-to Guides](#) section for details.

For more information about the framework, please refer to the [technical report](#) [MRT+25]. For clarifications on NVIDIA Isaac ecosystem, please check out the [Isaac Lab Ecosystem](#) section.



The Isaac Lab framework is open-sourced under the BSD-3-Clause license, with certain parts under Apache-2.0 license. Please refer to [License](#) for more details.

If you use Isaac Lab in your research, please cite our technical report:

```
@article{mittal2025isaacLab,  
  title={Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning},  
  author={Mayank Mittal and Pascal Roth and James Tigue and Antoine Richard and  
  journal={arXiv preprint arXiv:2511.04831},  
  year={2025},  
  url={https://arxiv.org/abs/2511.04831}  
}
```

Isaac Lab development initiated from the [Orbit](#) framework. We gratefully acknowledge the authors of Orbit for their foundational contributions.

Isaac Lab

[Isaac Lab Ecosystem](#)

[Local Installation](#)

[Container Deployment](#)

[Cloud Deployment](#)

[Reference Architecture](#)

Getting Started

[Quickstart Guide](#)

[Build your Own Project or Task](#)

[Create new project or task](#)

[Project Structure](#)

[Walkthrough](#)

[Environment Design Background](#)

[Classes and Configs](#)

[Environment Design](#)

[Training the Jetbot: Ground Truth](#)

[Exploring the RL problem](#)

[Tutorials](#)

[Creating an empty scene](#)

[Spawning prims into the scene](#)

Deep-dive into AppLauncher

Adding a New Robot to Isaac Lab

Interacting with a rigid object

Interacting with an articulation

Interacting with a deformable object

Interacting with a surface gripper

Using the Interactive Scene

Creating a Manager-Based Base Environment

Creating a Manager-Based RL Environment

Creating a Direct Workflow RL Environment

Registering an Environment

Training with an RL Agent

Configuring an RL Agent

Modifying an existing Direct RL Environment

Policy Inference in USD Environment

Adding sensors on a robot

Using a task-space controller

Using an operational space controller

How-to Guides

Importing a New Asset

Writing an Asset Configuration

Making a physics prim fixed in the simulation

Spawning Multiple Assets

Saving rendered images and 3D re-projection

Find How Many/What Cameras You Should Train With

Configuring Rendering Settings

Creating Visualization Markers

Wrapping environments

Adding your own learning library

Recording Animations of Simulations

Recording video clips during training

Curriculum Utilities

Mastering Omniverse for Robotics

Setting up CloudXR Teleoperation

Setting up Haply Teleoperation

Simulation Performance and Tuning

Optimize Stage Creation

Developer's Guide

Setting up Visual Studio Code

Configuring the python interpreter

Setting up formatting and linting

Repository organization

Extension Development

Overview

Core Concepts

Task Design Workflows

Actuators

Sensors

Camera

Contact Sensor

Frame Transformer

Inertial Measurement Unit (IMU)

Ray Caster

Visuo-Tactile Sensor

Motion Generators

Available Environments

Comprehensive List of Environments

Reinforcement Learning

Reinforcement Learning Scripts

Reinforcement Learning Library Comparison

Performance Benchmarks

Debugging and Training Guide

Imitation Learning

Teleoperation and Imitation Learning with Isaac Lab Mimic

Augmented Imitation Learning

SkillGen for Automated Demonstration Generation

Showroom Demos

Simple Agents

Features

Hydra Configuration System

Modifying advanced parameters

Modifying inter-dependent parameters

Multi-GPU and Multi-Node Training

Multi-GPU Training

Multi-Node Training

Population Based Training

What PBT Does

Leader / Underperformer Selection

Mutation (Hyperparameters)

Example Config

Launching PBT

Tips

Training Example

References

Tiled Rendering

Tiled Rendering

Annotators

RGB and RGBA

Depth and Distances

Normals

Motion Vectors

Semantic Segmentation

Instance ID Segmentation

Ray Job Dispatch and Tuning

Overview

Docker-based Local Quickstart

[Remote Clusters](#)

[Reproducibility and Determinism](#)

Experimental Features

[Welcome to the bleeding edge!](#)

[Newton Physics Integration](#)

[Installation](#)

[Installation](#)

[Testing the Installation](#)

[Isaac Lab 3.0 Beta](#)

[Training Environments](#)

[Visualization](#)

[Overview](#)

[Quick Start](#)

[Visualizer Backends](#)

[Performance Note](#)

[Limitations](#)

[Limitations](#)

[Solver Transitioning](#)

[Sim-to-Sim Policy Transfer](#)

[Overview](#)

[Sim-to-Real Policy Transfer](#)

[Requirements](#)

1. Train the teacher policy
2. Distill the student policy (remove privileged terms)
3. Fine-tune the student policy with RL

Resources

[Cloud Deployment](#)

[Sim2Real Deployment of Policies Trained in Isaac Lab](#)

Migration Guides

[From IsaacGymEnvs](#)

[From OmnitsaacGymEnvs](#)

[From Orbit](#)

[Source API](#)

[API Reference](#)

[References](#)

[Additional Resources](#)

[Contribution Guidelines](#)

[Tricks and Troubleshooting](#)

[Migration Guide \(Isaac Sim\)](#)

[Known Issues](#)

[Release Notes](#)

[Extensions Changelog](#)

[License](#)

[Bibliography](#)

- [Index](#)
- [Module Index](#)
- [Search Page](#)